

NUTZUNGSBEDINGUNGEN

=====

Dokumentstand: Aktiver Proof-of-Concept und Entwicklungsstand.

DevBox besitzt einen funktionsfähigen lokalen Launcher, eine grafische Hauptoberfläche, modulare Strukturansichten, Stammdaten- und Dokumentationspflege, Dokumentations-Snapshots, lokale Werkzeugerkennung, eine Repository-Seite sowie erste Bausteine für den DevBox-spezifischen Veröffentlichungsprozess.

Der deskNode-Bereich ist aktiv in DevBox integriert. Supervisor und Daemon können aus der GUI gestartet und gestoppt werden; deren Konsolenausgabe wird im deskNode-Log angezeigt. Die Produktversion kann aus den Stammdaten bearbeitet werden. UX-Themes können benannt, angelegt, gelöscht, dupliziert, umbenannt und gespeichert werden. Nach dem Speichern werden die deskNode-Produktdaten aktualisiert.

DeskNode verfügt im aktuellen Proof of Concept über eine gekoppelte Worker-Architektur. Tapo, FRITZ!DECT und Shelly werden lokal über eigene venmods entdeckt und überwacht; bei unterstützten Geräten werden Schaltzustände und Leistungswerte verarbeitet. Der Daemon synchronisiert die venmod-Daten in einen globalen Gerätebestand, hält einen initialen Sicherheitslesezyklus ein und startet die Hauptoberfläche erst nach der globalen Erst-Synchronisierung. Die Strukturverwaltung unterstützt mehrere Gebäude-Wurzeln, pro Gebäude einen vollständigen Geräte-Pool und zusätzliche additive Zuordnungen zu Räumen oder anderen Strukturgliedern.

Die Symbolverwaltung ist als funktionsfähige Katalogoberfläche vorhanden. Sie verwaltet deskNode-Geräte-kategorien und Verbrauchergeräte über Auswahlmenüs sowie Dialoge zum Anlegen, Bearbeiten und Löschen. Neue Geräte erhalten eine einzelne PNG-Quelle, die unter ihrer record_id als symbol_source_<record_id>.png abgelegt wird. Nach jeder erfolgreichen Katalogänderung wird create_manufacturer_db.py ausgeführt, damit mnfctr_db.r0b die aktuellen Theme-, Kategorien- und Gerätedaten enthält.

Der Graphic-Pack-Build ist als Proof of Concept funktionsfähig. Er erzeugt aus Symbolquellen und UX-Themes vorbereitete Grafikzustände für Verbraucher. Die aktuelle deskNode-Laufzeit nutzt bereits Sprach-, Theme- und Grafikdaten; Asset-Auswahl, weitere Symbolabdeckung und visuelle Zustände werden weiterentwickelt.

DevBox und deskNode sind keine fertigen Endnutzer- oder Release-Versionen. Besonders Credential-Speicherung, Langzeitstabilität größerer Gerätebestände, herstellersistenspezifische Sonderfälle, Tests, Packaging und Plattformportierung sind noch offene Entwicklungsaufgaben.

Erstveröffentlichung: Autor / Herausgeber: Markus Walloner Land: Germany (DE)

1. Gegenstand

Diese Nutzungsbedingungen beziehen sich auf die in der begleitenden Projektdokumentation beschriebene Software bzw. das beschriebene Projekt.

Kurzbeschreibung: DevBox ist die lokale Entwicklungszentrale der CYXnTrol Development Platform. Die PySide6-Oberfläche bündelt Projektstruktur, Stammdaten, Dokumentation, lokale Werkzeuge, Produktmodule, Repository-Vorbereitung und wiederkehrende Entwicklungsabläufe.

Mit deskNode ist ein aktives Produktmodul integriert: ein lokaler Geräte- und Strukturhub für Smart-Plugs und angeschlossene Verbraucher. deskNode wird über austauschbare venmods für unterstützte Hersteller und Protokolle erweitert.

DevBox ist kein Endnutzerprodukt. Es ist eine interne Arbeitsumgebung, in der Anforderungen, Datenmodelle, UX-Entscheidungen, Schnittstellen, Builds und Veröffentlichungsstände nachvollziehbar vorbereitet und geprüft werden.

Langbeschreibung: DevBox verbindet Aufgaben, die sonst über Einzelskripte, Ordner, Tabellen, Konsolenfenster und externe Programme verteilt wären. Die Anwendung verwaltet zentrale Projekt- und Produktdaten, stellt Dokumentationsinhalte bereit, organisiert globale Strukturvorlagen, erkennt lokale Werkzeuge und bündelt produktbezogene Entwicklungsfunktionen in einer gemeinsamen Oberfläche.

Das erste aktive Produktmodul ist deskNode. deskNode ist eine lokale Steuer- und Strukturumgebung für Smart-Plugs und angeschlossene Verbraucher. Ein Supervisor startet einen Daemon; dieser koppelt gefundene venmods über einen gemeinsamen Vertrag ein und startet pro venmod einen eigenen Worker. Der aktuelle Proof of Concept enthält lokale Pfade für Tapo, FRITZ!DECT und Shelly. Geräte werden in einer globalen Inventardatenbank geführt, während jeder Worker nur seine eigene flüchtige Laufzeitdatenbank schreibt. Der Daemon synchronisiert die bestätigten Livewerte, Sollzustände und Geräteidentitäten zwischen den Ebenen.

deskNode organisiert Geräte zusätzlich in unabhängigen Gebäudebäumen. Jedes Gebäude besitzt einen Geräte-Pool als vollständige Gebäudeübersicht sowie optionale räumliche oder funktionale Strukturglieder. Ein Gerät kann im Pool eines Gebäudes bleiben und zusätzlich einem Raum, Bereich, Desk oder anderen Strukturglied zugeordnet sein. Die grafische Oberfläche visualisiert diese Struktur, Geräte- und Leistungswerte sowie lokale Schaltzustände.

Die DevBox-Seite für deskNode startet und stoppt Supervisor und Daemon, zeigt deren Konsolenausgabe, pflegt Produktversionen, UX-Themes und den Symbolkatalog. Themes werden mit benannten Datensätzen, RGBA-Farben, globalen Schriftdateien, Größen, Konturen und Formregeln gepflegt. Kategorien und Verbrauchersymbole werden über stabile technische IDs und PNG-Quellen verwaltet. Der Graphic-Pack-Build erzeugt daraus vorbereitete Varianten für Themes und Zustände.

DevBox arbeitet lokal im Projektkontext. Ein Launcher ermittelt den Projektroot über die Datei .root, stellt Laufzeitdaten unter AppData bereit, prüft externe Werkzeuge und startet die GUI aus einer temporären Arbeitskopie. Der echte Projektroot bleibt die maßgebliche Quelle für Ressourcen, Datenbanken und Funktionsskripte. Eine Einzelinstanzlogik verhindert parallele DevBox-Fenster und aktiviert beim erneuten Start die bereits geöffnete Instanz.

Das Projekt entsteht in einem KI-gestützten, iterativen Entwicklungsprozess. Fachliche Anforderungen, Prozesslogik, Datenmodelle, UX-Entscheidungen, Testfälle und Abnahmen

werden durch den Projektverantwortlichen gesteuert; Implementierungsschritte werden mit KI-gestützten Werkzeugen erstellt, überprüft und fortlaufend nachgeschärft.

2. Nutzungsbedingungen

NUTZUNGSBEDINGUNGEN – ENTWURFSFASSUNG

1. Geltungsbereich

Diese Nutzungsbedingungen gelten für DevBox, die lokale Entwicklungsumgebung der CYXnTrol Development Platform. DevBox ist ein Entwicklungswerkzeug und keine fertige produktive Endnutzeranwendung.

2. Entwicklungsstand

DevBox und das darin integrierte Produktmodul deskNode befinden sich in aktiver Entwicklung. Funktionen, Schnittstellen, Datenbankstrukturen, venmods, Grafik-Builds, Ordnerstrukturen, Dokumente und Repository-Prozesse können sich ändern, unvollständig sein oder Fehler enthalten. Die Nutzung erfolgt auf eigene Verantwortung.

3. Zulässige Nutzung

DevBox darf zur lokalen Organisation, Dokumentation, Entwicklung und Pflege eigener Softwareprojekte oder Projekte verwendet werden, für die eine entsprechende Berechtigung vorliegt. Nutzer sind dafür verantwortlich, dass sie die notwendigen Rechte an verarbeiteten Inhalten, Daten, Quellcode, Bildern, Fontdateien, Repositories, Geräten und externen Diensten besitzen.

4. deskNode und Gerätesteuerung

DeskNode kann in seinem Entwicklungsstand lokale Geräte erkennen, Zustände auslesen und unterstützte Verbraucher schalten. Eine Schaltfunktion darf nicht als Sicherheitsfunktion, Not-Aus, Schutzabschaltung, medizinische Steuerung oder alleinige Überwachung kritischer Infrastruktur verwendet werden. Der Nutzer bleibt für sichere Konfigurationen, lokale Netzwerksicherheit, physische Schutzmaßnahmen, Gerätekompatibilität und die Folgen eines Schaltvorgangs verantwortlich.

5. Zugangsdaten

Zugangsdaten für unterstützte Herstellerpfade werden nur lokal verarbeitet. Der aktuelle Entwicklungsstand besitzt noch keinen plattformübergreifend verschlüsselten Credential-Vault. Bis zu dessen Umsetzung dürfen keine besonders schutzbedürftigen oder produktionskritischen Zugangsdaten allein auf diesen Stand gestützt werden.

6. Externe Programme und Dienste

DevBox kann nach ausdrücklicher Nutzeraktion lokal installierte Programme wie Inkscape, GIMP, Git, Git Credential Manager und Installer starten oder mit ihnen zusammenarbeiten. Für diese Programme, ihre Lizenzen, Updates, Datenverarbeitung und Nutzungsbedingungen sind die jeweiligen Anbieter oder Rechteinhaber verantwortlich.

Ein Repository-Push, Grafik-Build oder produktbezogener Prozess wird nur nach ausdrücklicher Nutzeraktion vorbereitet oder gestartet. Der Nutzer ist verantwortlich für Zielpfade, Repository-URL, Branch, Sichtbarkeit, Commit-Inhalte und Veröffentlichungsberechtigung.

7. Keine Gewähr

Soweit rechtlich zulässig wird DevBox ohne Zusicherung von Verfügbarkeit, Fehlerfreiheit, Eignung für einen bestimmten Zweck oder vollständiger Kompatibilität bereitgestellt. DevBox ersetzt insbesondere keine Backups, Rechtsprüfung, Sicherheitsprüfung, fachliche Abnahme oder Release-Freigabe.

8. Schutz des Entwicklungsstands

DevBox verwendet temporäre Arbeitsbereiche für Export-, Grafik-, Runtime- und Veröffentlichungsoperationen. Dennoch bleibt der Nutzer verantwortlich, aktuelle Sicherungen des Projektstands zu erstellen und Zielorte, Datenbankänderungen, Cleanup-Operationen, Git-Vorgänge, Gerätebefehle und Toolstarts vor ihrer Ausführung zu prüfen.

9. Dokumente und rechtliche Inhalte

Mit DevBox erzeugte README-, Lizenz-, Datenschutz- oder Nutzungsdokumente können technische Vorlagen oder Entwürfe enthalten. Vor öffentlicher oder kommerzieller Nutzung müssen sie für das konkrete Produkt, dessen tatsächliche Datenflüsse, Vertriebswege, Vertragsbeziehungen und das anwendbare Recht geprüft und gegebenenfalls angepasst werden.

10. Haftungsbeschränkung

Soweit rechtlich zulässig haftet der Autor nicht für mittelbare Schäden, Datenverlust, entgangenen Nutzen, Fehlkonfigurationen, fehlerhafte Veröffentlichungen, fehlerhafte Asset-Builds, fehlerhafte Geräteschaltungen oder Folgen der Nutzung externer Programme und Dienste. Zwingende gesetzliche Ansprüche bleiben unberührt.

11. Änderungen

Diese Entwurfsfassung kann mit der Weiterentwicklung von DevBox angepasst werden.

3. Lizenzbezug

Lizenz: Zero-Clause BSD License 0BSD

Die vollständigen Lizenzbedingungen sind in der zugehörigen Lizenzdatei hinterlegt.

4. Projektbezogene Hinweise

Zweck: DevBox soll wiederkehrende und fehleranfällige Entwicklungsarbeit in nachvollziehbare lokale Werkzeuge überführen. Dazu gehören insbesondere die Pflege von Produkt- und Herstellerdaten, strukturierte Dokumentation, vorbereitete

Veröffentlichungsschritte, die Integration lokaler Kreativprogramme sowie die kontrollierte Ausführung produktbezogener Entwicklungsfunktionen.

Die Anwendung schafft eine gemeinsame Arbeitsgrundlage für Produkte der CYXnTrol Development Plattform. Statt Abläufe nur als Erinnerung, Ordnerkonvention oder Sammlung einzelner Konsolenbefehle zu halten, werden sie als Daten, Skripte, GUI-Funktionen und überprüfbare Prozessketten festgehalten.

Für deskNode dient DevBox als Entwicklungs- und Konfigurationsoberfläche für Produktversionen, UX-Themes, Gerätekategorien, Verbrauchersymbole, Sprachressourcen und vorbereitete Grafikpakete. Die deskNode-Laufzeit soll daraus reproduzierbare Daten und vorbereitete Assets erhalten. Die venmod-Architektur soll zudem neue lokale Gerätepfade ergänzen können, ohne den Daemon oder bereits funktionierende Herstellerpfade unnötig umzubauen.

Kontext: Die CYXnTrol Development Plattform entstand aus praktischer Entwicklungsarbeit mit vielen kleinen Werkzeugen, Datenständen, Grafikdateien, Dokumenten und Testabläufen. Mit wachsender Zahl von Produkten und Funktionen reicht es nicht mehr aus, Informationen nur in einzelnen Dateien oder im Gedächtnis zu halten. Benötigt werden eine stabile Projektwurzel, nachvollziehbare Datenquellen, wiederholbare temporäre Arbeitsbereiche und klar abgegrenzte Produktmodule.

DevBox ist die Antwort auf diesen Bedarf. Es bildet eine lokale Entwicklungszentrale, in der Hersteller- und Produktdaten, Dokumentation, globale Strukturregeln, externe Werkzeuge, Repository-Vorbereitung und spezielle Produktfunktionen zusammengeführt werden.

deskNode ist das erste Produkt, das als eigener aktiver Bereich in DevBox integriert wird. Seine Aufgabe ist die lokale Verwaltung, Visualisierung und Schaltung unterstützter Smart-Plugs. Tapo, FRITZ!DECT und Shelly werden nicht als fest in die Oberfläche eingebaute Sonderfälle behandelt, sondern über venmods mit einem gemeinsamen Coupler- und Worker-Vertrag angebunden. Für Verbraucher werden Namen, Kategorien, stabile technische IDs, PNG-Quellen, Theme-Daten und vorberechnete Grafikzustände in einen reproduzierbaren Ablauf gebracht.

Repository-Hinweis: Das DevBox-Repository repräsentiert die CYXnTrol Development Plattform selbst, nicht ein fertiges Endnutzerprodukt. Es dient als Entwicklungsstand, transparente Arbeitsprobe und Grundlage für die Pflege der Plattform.

Der veröffentlichte Stand wird nicht direkt aus dem produktiven Projektroot kopiert. Für DevBox wird ein gesonderter bereinigter root_dir-Stand erzeugt. Er enthält vorgesehenen Quellcode, Dokumente und Assets, während lokale Laufzeitdaten, temporäre Reste, Installer, Datenbank-WAL-Dateien und nicht veröffentlichte Daten außerhalb des Veröffentlichungspakets bleiben.

Der derzeitige DevBox-Push behandelt applications als bewusstes Auslieferungsfenster: Der Bereich wird zunächst bereinigt. Danach wird applications/deskNode/resources/scripts einschließlich enhaltener Dateien in den Veröffentlichungsstand kopiert. __pycache__-Ordner sowie .pyc- und .pyo-Dateien bleiben ausgeschlossen. Dieser aktuelle Umfang ist ein DevBox-spezifischer Veröffentlichungsprozess und noch kein vollständiger deskNode-Produktrelease. Bevor deskNode einschließlich GUI, Logik und venmods als eigenes Repository oder Release veröffentlicht wird, muss dessen Exportumfang

ausdrücklich definiert und getestet werden.

DevBox wird in einem KI-gestützten, iterativen Workflow entwickelt. Anforderungen, Architektur, Datenmodelle, UX-Entscheidungen, Testfälle und Abnahmen werden durch den Projektverantwortlichen gesteuert. Die Implementierung entsteht mit KI-gestützten Entwicklungswerkzeugen und wird gegen die definierten Funktionsanforderungen geprüft.

Copyright (c) 2026 Markus Walloner